A - Lucky Direction

Time Limit: 2 sec / Memory Limit: 1024 MB

Problem Statement

You are given a string D representing one of the eight directions (north, east, west, south, northeast, northwest, southeast, southwest). The correspondence between the directions and their representing strings is as follows.

- North: `N`
- East: `E`
- West: `W`
- South: `S`
- Northeast: `NE`
- Northwest: `NW`
- Southeast: `SE`
- Southwest: `SW`

Print the string representing the direction opposite to the direction denoted by D.

给你一个字符串 D 代表八个方向（北、东、西、南、东北、西北、东南、西南）之一。方向与其代表字符串之间的对应关系如下。

- 北`N`
- 东：`E
- 西： `W`
- 南： `S`
- 东北： `NE`
- 西北： `NW`
- 东南：`SE
- 西南： `SW`

打印与 D 表示的方向相反的字符串。

Constraints

- D is one of `N`, `E`, `W`, `S`, `NE`, `NW`, `SE`, `SW`.
- D 是 `N`、`E`、`W`、`S`、`NE`、`NW`、`SE`、`SW` 中的一个。

题解

将 D 中的 `N` 替换为 `S` ， `S` 替换为 `N` ， `E` 替换为 `W` ， `W` 替换为 `E` 即可。

```
s = input()
ns = ''
for i in s:
    if i == 'N':
        ns += 'S'
    elif i=='S':
        ns += 'N'
    elif i=='W':
        ns += 'E'
    else:
        ns += 'W'
print(ns)
```

B - Seek Grid

Time Limit: 2 sec / Memory Limit: 1024 MB

Problem Statement

You are given an $N \times N$ grid S and an $M \times M$ grid T. The cell at the i-th row from the top and the j-th column from the left is denoted by (i, j).

The colors of the cells in S and T are represented by N^2 characters $S_{i,j}$ $(1 \leq i, j \leq N)$ and M^2 characters $T_{i,j}$ $(1 \leq i, j \leq M)$, respectively. In grid S, cell (i, j) is white if $S_{i,j}$ is `.`, and black if $S_{i,j}$ is `#`. The same applies for grid T.

Find T within S. More precisely, output integers a and b $(1 \leq a, b \leq N - M + 1)$ that satisfy the following condition:

- $S_{a+i-1,b+j-1} = T_{i,j}$ for every i, j $(1 \leq i, j \leq M)$.

给你一个 $N \times N$ 网格 S 和一个 $M \times M$ 网格 T 。位于从上往下第 i 行和从左往上第 j 列的单元格用 (i, j) 表示。

S 和 T 中单元格的颜色分别用 N^2 字符 $S_{i,j}$ （$1 \leq i, j \leq N$）和 M^2 字符 $T_{i,j}$ （$1 \leq i, j \leq M$）表示。在网格 S 中，如果 $S_{i,j}$ 为"."，则单元格 (i, j) 为白色；如果 $S_{i,j}$ 为 "#"，则单元格 (i, j) 为黑色。网格 T 也是如此。

在 S 中找到 T 。更精确地说，输出满足以下条件的整数 a 和 b （$1 \leq a, b \leq N - M + 1$）：

- 每个 i, j ($1 \leq i, j \leq M$) 都有 $S_{a+i-1,b+j-1} = T_{i,j}$ 个整数。

Constraints

- $1 \leq M \leq N \leq 50$
- N and M are integers.
- N 和 M 都是整数。
- Each of $S_{i,j}$ and $T_{i,j}$ is `.` or `#`.
- $S_{i,j}$ 和 $T_{i,j}$ 中的每一个都是 `.` 或 `#`。
- There is exactly one pair (a, b) satisfying the condition.
- 正好有一对 (a, b) 满足条件。

题解

暴力遍历所有可能得左上角，如果匹配则输出。

```
def solve():
    n, m = map(int, input().split())
    gs = [list(input()) for i in range(n)]
    gt = [list(input()) for i in range(m)]
    for bx in range(n-m+1):
        for by in range(n-m+1):
            f = 1
            for x in range(m):
                for y in range(m):
                    if gt[x][y] != gs[bx+x][by+y]:
                        f = 0
                        break
                if not f:
                    break
            if f:
                print(bx+1, by+1)
                return
solve()
```

C - Pigeonhole Query

Time Limit: 2 sec / Memory Limit: 1024 MB

Problem Statement

There are N pigeons numbered from 1 to N, and there are N nests numbered from 1 to N. Initially, pigeon i is in nest i for $1 \leq i \leq N$.

You are given Q queries, which you must process in order. There are two types of queries, each given in one of the following formats:

- `1 P H` : Move pigeon P to nest H.
- `2` : Output the number of nests that contain more than one pigeon.

有 N 只鸽子，编号从 1 到 N ，有 N 个鸽巢，编号从 1 到 N 。最初，鸽子 i 在 $1 \leq i \leq N$ 的巢 i 中。

您会收到 Q 个查询，您必须按顺序处理这些查询。查询有两种类型，每种都以下列格式之一给出：

- `1 P H` : 将鸽子 P 移至鸽巢 H 。
- `2` : 输出包含一只以上鸽子的鸽巢数量。

Constraints

- $2 \leq N \leq 10^6$
- $1 \leq Q \leq 3 \times 10^5$
- $1 \leq P, H \leq N$
- For a query of the first type, pigeon P is not in nest H before the move.
- All input values are integers.

- 对于第一种类型的查询，鸽子 P 在移动前不在鸽巢 H 中。
- 所有输入值均为整数。

题解

我们维护下两个信息

- 每只鸽子在那个鸽巢
- 每只鸽巢有几只鸽子

这可以用数组或map完成

为高效统计包含一只以上鸽子的鸽巢数量，每次查询时不去遍历所有的鸽巢。有以下关系

- 当有鸽巢中的鸽子的数量由 2 变为 1 后，满足条件的鸽巢数量 -1。
- 当有鸽巢中的鸽子的数量由 1 变为 2 后，满足条件的鸽巢数量 $+1$。

只需要在某个鸽巢的鸽子的数量发生变化时判断包含一只以上鸽子的鸽巢数量有无变化即可

```
n, q = map(int, input().split())
nest = {i:1 for i in range(1, n+1)} # i鸽巢有几只鸽子
check = {i:i for i in range(1, n+1)} # i鸽子在那个鸽巢
number = 0

for _ in range(q):
    cmd = list(map(int, input().split()))
    if cmd[0] == 1:
        p, h = cmd[1:]
        nest[check[p]] -= 1
        if nest[check[p]] == 1:  # 鸽子的数量由2变为1
            number -=1
        check[p] = h # 换鸽巢
        nest[check[p]] += 1
        if nest[check[p]] == 2:  # 鸽子的数量由1变为2
            number +=1
    else:
        print(number)
```

D - Gravity

Time Limit: 2 sec / Memory Limit: 1024 MB

Problem Statement

There is a grid with 10^9 rows and W columns. The cell at the x-th column from the left and the y-th row from the **bottom** is denoted by (x, y).

There are N blocks. Each block is a 1×1 square, and block i-th ($1 \leq i \leq N$) is located at cell (X_i, Y_i) at time 0.

At times $t = 1, 2, \ldots, 10^{100}$, the blocks are moved according to the following rules:

- If the entire bottom row is filled with blocks, then all blocks in the bottom row are removed.
- For each remaining block, in order from bottom to top, perform the following:

- If the block is in the bottom row, or if there is a block in the cell immediately below it, do nothing.
- Otherwise, move the block one cell downward.

You are given Q queries. For the j-th query ($1 \leq j \leq Q$), answer whether block A_j exists at time $T_j + 0.5$.

有一个网格，网格中有 10^9 行和 W 列。从左边起第 x 列和从**底部起第 y 行的单元格用 (x, y) 表示。

共有 N 个图块。每个图块是一个 1×1 正方形，图块 i （ $1 \leq i \leq N$ ）位于时间 0 的 (X_i, Y_i) 单元格。

在时间 $t = 1, 2, \ldots, 10^{100}$ 时，这些图块会根据以下规则移动：

- 如果整个底行都布满了图块，那么底行的所有图块都会被移走。
- 对于剩余的每个图块，按照从下到上的顺序，执行以下操作：
  - 如果该图块位于最下面一行，或者它下面的单元格中有一个图块，则不做任何操作。
  - 否则，将该图块向下移动一格。

给你 Q 个查询。对于 j -查询（ $1 \leq j \leq Q$ ），请回答在 $T_j + 0.5$ 时间块 A_j 是否存在。

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq W \leq N$
- $1 \leq X_i \leq W$
- $1 \leq Y_i \leq 10^9$
- $(X_i, Y_i) \neq (X_j, Y_j)$ if $i \neq j$.
- $1 \leq Q \leq 2 \times 10^5$
- $1 \leq T_j \leq 10^9$
- $1 \leq A_j \leq N$
- All input values are integers.
- 所有输入值均为整数。

题解

我们定义 $lvl_{x,y} = i$ 为对于 x 相同的点中，当前点的 y 是第 i 小 。

由题意可知每次被消除的点的 lvl 一定相同。

而每一 lvl 被消除要满足

- 这一层有 w 个点
- 所有点都到了 $y = 1$

不难想到， 每一 lvl 被消除的时间取决于这一 lvl 的点中 y 的最大值。

对每一 lvl ，最上方的点到达 $y = 1$ 的时间为 $y_i - 1$， 下一秒这一层的点被消除。

又有 第 lvl 层的点至少在 $lvl - 1$ 层消除后一秒才会被消除。

我们预处理每个点在那一秒被消除了，之后 $O(1)$ 回答每个询问即可。

```python
from collections import defaultdict

n, w = map(int, input().split())
ans  = [10**9+7 for i in range(n+1)]
place = defaultdict(list)
level = defaultdict(list)
mxlvl = 0
for _ in range(n):
    x, y = map(int, input().split())
    place[x].append([y,_])
for nx in place:
    place[nx].sort()
for nx in place:
    mxlvl = max(mxlvl, len(place[nx]))
    for lvl in range(len(place[nx])):
        level[lvl+1].append(place[nx][lvl])
pre = 0
for lv in range(1, mxlvl + 1):
    if len(level[lv]) == w:
        mxy = max(level[lv], key=lambda x:x[0])[0]
        for y, idx in level[lv]:
            ans[idx] = max(mxy, pre+1)
        pre = mxy
q = int(input())
for _ in range(q):
    t, a = map(int, input().split())
    t += 0.5
    a -= 1
    if t > ans[a]:
        print('No')
    else:
        print('Yes')
```

E - Hierarchical Majority Vote

Time Limit: 2 sec / Memory Limit: 1024 MB

Problem Statement

For a binary string $B = B_1B_2\ldots B_{3^n}$ of length 3^n ($n \geq 1$), we define an operation to obtain a binary string $C = C_1C_2\ldots C_{3^{n-1}}$ of length 3^{n-1} as follows:

- Partition the elements of B into groups of 3 and take the majority value from each group. That is, for $i = 1, 2, \ldots, 3^{n-1}$, let C_i be the value that appears most frequently among B_{3i-2}, B_{3i-1}, and B_{3i}.

You are given a binary string $A = A_1A_2\ldots A_{3^N}$ of length 3^N. Let $A' = A'_1$ be the length-1 string obtained by applying the above operation N times to A.

Determine the minimum number of elements of A that must be changed (from 0 to 1 or from 1 to 0) in order to change the value of A'_1.

对于长度为 3^n ($n \geq 1$) 的二进制字符串 $B = B_1B_2\ldots B_{3^n}$ ，我们定义了如下操作来获取长度为 3^{n-1} 的二进制字符串 $C = C_1C_2\ldots C_{3^{n-1}}$ ：

- 将 B 中的元素分成 3 组，并从每组中取多数值。也就是说，对于 $i = 1, 2, \ldots, 3^{n-1}$ ，让 C_i 成为 B_{3i-2} 、 B_{3i-1} 和 B_{3i} 中出现频率最高的值。

给你一个长度为 3^N 的二进制字符串 $A = A_1 A_2 \ldots A_{3^N}$ 。假设 $A' = A'_1$ 是对 A 进行 N 次上述运算后得到的长度为 1 的字符串。

求要改变 A'_1 的值，必须改变 A 中最少多少个元素（从 0 改为 1 或从 1 改为 0 ）。

- N is an integer with $1 \leq N \leq 13$.
- A is a string of length 3^N consisting of 0 and 1.

Constraints

- N is an integer with $1 \leq N \leq 13$.
- N 是包含 $1 \leq N \leq 13$ 的整数。
- A is a string of length 3^N consisting of 0 and 1.
- A 是长度为 3^N 的字符串，由 0 和 1 组成。

题解

不妨先思考 $A = 000$ 时的情况

易得原来的答案为 0 ，要想变为 1，我们至少要改变两个 0 为 1，改变每个 0 的代价为 1

我们维护每一次操作过程中每一个组的值，以及更改值的代价。

假如某个组为 000 , 更改时选取代价最小的两个 0

假如某个组为 001 , 更改时选取代价最小的 0

以此类推我们可以得到最终的最小代价

```
n = int(input())
s = list(input())

dp = [[0, 0] for i in range(3**(n-1))] # dp[i][0] 为组的值，dp[i][1] 为更改值的最小代价
for i in range(0, len(s), 3):
    c = [0, 0]
    c[int(s[i])] += 1
    c[int(s[i+1])] += 1
    c[int(s[i+2])] += 1
    dp[i//3][0] = 1 if c[1] > c[0] else 0
    dp[i//3][1] = 1 if abs(c[1]-c[0]) == 1 else 2
s = n-2
for r in range(n-1):
    ndp = [[0, 0] for i in range(3**s)]
    for i in range(0, len(dp), 3):
        c = [0, 0]
        cost = [[], []]
        for j in range(3):
            c[dp[i+j][0]] += 1
            cost[dp[i+j][0]].append(dp[i+j][1])
        ndp[i//3][0] = 1 if c[1] > c[0] else 0
        n = len(cost[0])
        m = len(cost[1])
```

```
        k = cost[ndp[i//3][0]]
        k.sort()
        need = (max(n, m) - min(n, m) + 1)//2
        ndp[i//3][1] = sum(k[:need])
    dp = ndp
    s -= 1
print(dp[0][1])
```

F - K-th Largest Triplet

Time Limit: 3 sec / Memory Limit: 1024 MB

Problem Statement

You are given three integer sequences of length N, namely $A = (A_1, A_2, \ldots, A_N)$, $B = (B_1, B_2, \ldots, B_N)$, and $C = (C_1, C_2, \ldots, C_N)$, and an integer K.

For each of the N^3 choices of integers i, j, k ($1 \leq i, j, k \leq N$), compute the value $A_iB_j + B_jC_k + C_kA_i$. Among all these values, find the K-th largest value.

给你三个长度为 N 的整数序列，即 $A = (A_1, A_2, \ldots, A_N)$ 、 $B = (B_1, B_2, \ldots, B_N)$ 和 $C = (C_1, C_2, \ldots, C_N)$ 以及一个整数 K 。

对于 N^3 中的每一个整数 i, j, k （ $1 \leq i, j, k \leq N$ ），计算其值 $A_iB_j + B_jC_k + C_kA_i$ 。在所有这些值中，找出 第 K 大的值。

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq K \leq \min(N^3, 5 \times 10^5)$
- $1 \leq A_i, B_i, C_i \leq 10^9$
- All input values are integers.
- 所有输入值均为整数。

题解

观察到 $K \leq 5 \times 10^5$，因此算出 K 个最大值是可行的。

我们可以用类似 BFS 的方式来进行。

首先对 A, B, C 降序排序。

初始我们将 $f(0,0,0), 0, 0, 0$ 放入队列中

每一次，我们取出队列中的最大值和对应的 i, j, k，并尝试$i+1, j, k$、$i, j+1, k$和$i, j, k+1$。

累计执行了 K 次后即得到了答案。

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
```

```
const ll INF = INT_MAX;
const ll MOD = 1000000007;
const ll mod = 998244353;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}
ll encode(ll i, ll j, ll k){
    return (i<<40)|(j<<20)|k;
}

bool cmp(ll x, ll y){
    return x > y;
}

void printVector(vector<ll> vec){
    for(const auto t:vec){
        pt(t),putchar(' ');
    }
    puts("");
}

void solve(){
    ll n = read(), k = read();
    vector<ll> a(n), b(n), c(n);
    for(ll i=0;i<n;i++) a[i] = read();
    for(ll i=0;i<n;i++) b[i] = read();
    for(ll i=0;i<n;i++) c[i] = read();
    sort(a.begin(), a.end(), cmp);
    sort(b.begin(), b.end(), cmp);
    sort(c.begin(), c.end(), cmp);
    auto f = [&](ll i, ll j, ll k){
        return a[i]*b[j] + a[i]*c[k] + b[j]*c[k];
    };
    priority_queue<array<ll, 4>> pq;
    pq.push({f(0, 0, 0), 0, 0, 0});

    ll ans = 0;
    unordered_set<ll> used;
    used.insert(encode(0, 0, 0));
    while(k--){
        auto [val, i, j, k] = pq.top();
        pq.pop();
        ans = val;
        if(i + 1< n){
            ll code = encode(i+1, j, k);
            if(used.find(code)==used.end()){
```

```
                used.insert(code);
                pq.push({f(i+1, j, k), i+1, j, k});
            }
        }
        if(j + 1< n){
            ll code = encode(i, j+1, k);
            if(used.find(code)==used.end()){
                used.insert(code);
                pq.push({f(i, j+1, k), i, j+1, k});
            }
        }
        if(k + 1< n){
            ll code = encode(i, j, k + 1);
            if(used.find(code)==used.end()){
                used.insert(code);
                pq.push({f(i, j, k+1), i, j, k + 1});
            }
        }
    }
    print(ans);
}

int main(){
    ll t = 1;
    //t = read();
    while(t--){
        solve();
    }
}
```

~~G Many LCS (还没做出来)~~